UNIT 1 OPERATING SYSTEM OVERVIEW

Introduction to OS - Functionality of OS - Evolution of Operating System-Operating System Structuring methods(monolithic, layered, modular, micro-kernel models)-system calls-system programs- Hardware Protection user/kernel modes-Memory Hierarchy, Cache Memory, Multiprocessor and Multicore Organization.

1. INTRODUCTION TO OPERATING SYSTEM

BASIC ELEMENTS OF A COMPUTER:

A computer consists of processor, memory, I/O components and system bus.

i) **Processor**: It Controls the operation of the computer and performs its data processing functions. When there is only one processor, it is often referred to as the central processing unit.

ii) **Main memory**: It Stores data and programs. This memory is typically volatile; that is, when the computer is shut down, the contents of the memory are lost. Main memory is also referred to as real memory or primary memory.

iii) **I/O modules**: It moves data between the computer and its external environment. The external environment consists of a variety of devices, including secondary memory devices (e. g., disks), communications equipment, and terminals.

iv) **System bus**: It provides the communication among processors, main memory, and I/O modules.

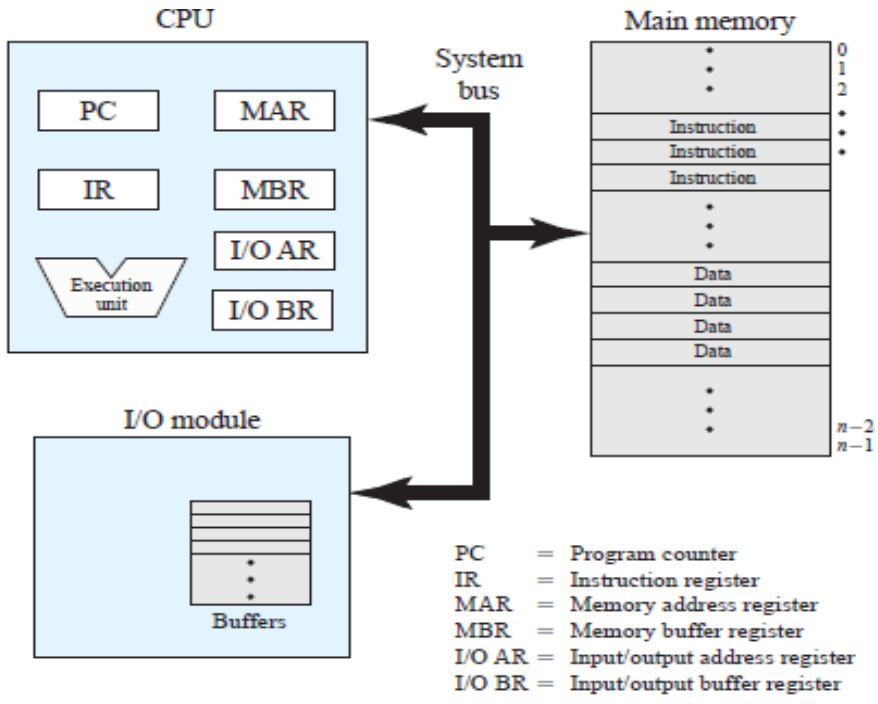


Figure 1.1 **Computer Components: Top-Level View**

- One of the processor's functions is to exchange data with memory. For this purpose, it typically makes use of two internal registers

i) **A memory address registers (MAR),** which specifies the address in memory for the next read or write.

ii) **A memory buffer register (MBR)**, which contains the data to be written into memory or which receives the data read from memory.

- **An I/O address register (I/OAR)** specifies a particular I/O device.
- **An I/O buffer register (I/OBR)** is used for the exchange of data between an I/O module and the processor.
- **A memory module** consists of a set of locations, defined by sequentially numbered addresses.
- **An I/O module** transfers data from external devices to processor and memory, and vice versa. It contains internal buffers for temporarily holding data until they can be sent on.

PROCESSOR REGISTERS:

A processor includes a set of registers that provide memory that is faster and smaller than main memory. Processor registers serve two functions:

i) **User-visible registers:** Enable the machine or assembly language programmer to minimize main memory references by optimizing register use.

ii) **Control and status registers**: Used by the processor to control the operation of the processor and by privileged OS routines to control the execution of programs.

1. User-Visible Registers:

A user-visible register is generally available to all programs, including application programs as well as system programs. The types of User visible registers are

i) Data Registers

ii) Address Registers

Data Registers can be used with any machine instruction that performs operations on data. **Address registers** contain main memory addresses of data and instructions. Examples of address registers include the following:

• Index register.

• Segment pointer

• Stack pointer

2. **Control and status register:**

A variety of processor registers are employed to control the operation of the processor. In addition to the MAR, MBR, I/OAR, and I/OBR register the following are essential to instruction execution:

• **Program counter (PC):** Contains the address of the next instruction to be fetched.

• **Instruction register (IR):** It contains the instruction most recently fetched.

All processor designs also include a register or set of registers, often known as the program

status word (PSW) that contains status information. The PSW typically contains condition codes plus other status information, such as an interrupt enable/disable bit and a kernel/user mode bit, carry bit, auxiliary carry bit.

COMPUTER SYSTEM ORGANIZATION:

- Computer system organization deals with the structure of the computer system.

Computer system operation:

- A modern general-purpose computer system consists of one or more CPUs and a number of device controllers connected through a common bus that provides access to shared memory.
- For a computer to start running when it is powered up or rebooted—it needs to have an initial program to run. This initial program is called as the Bootstrap program.
- It is stored within the computer hardware in read-only memory (**ROM**) or electrically erasable programmable read-only memory (**EEPROM**), known by the general term **firmware.**
- The bootstrap loader It initializes all aspects of the system, from CPU registers to device controllers to memory contents.
- The bootstrap program loads the operating system and start executing that system.
- Once the kernel is loaded and executing, it can start providing services to the system and its users. When is the system is booted it waits for some event to occur.
- The occurrence of an event is usually signaled by an **interrupt** from either the hardware or the software.
- When the CPU is interrupted, it stops what it is doing and immediately transfers execution to a fixed location. That contains the starting address of the service routine for the interrupt.
- The interrupt service routine executes; on completion, the CPU resumes the interrupted computation.

Storage structure:

- The CPU can load instructions only from memory, so any programs to run must be stored in main memory.
- Main memory commonly is implemented in a semiconductor technology called **dynamic random-access memory**
- ROM is a read only memory that is used to store the static programs such as bootstrap loader.
- All forms of memory provide an array of bytes. Each byte has its own address. The operations are done through load or store instructions.
- The load instruction moves a byte or word from main memory to an internal register within the CPU, whereas the store instruction moves the content of a register to main memory.

❖ Ideally, we want the programs and data to reside in main memory permanently. This arrangement usually is not possible for the following two reasons

i) Main memory is usually too small to store all needed programs and data permanently

ii) Main memory is a **volatile** storage device that loses its contents when power is turned off or otherwise lost.

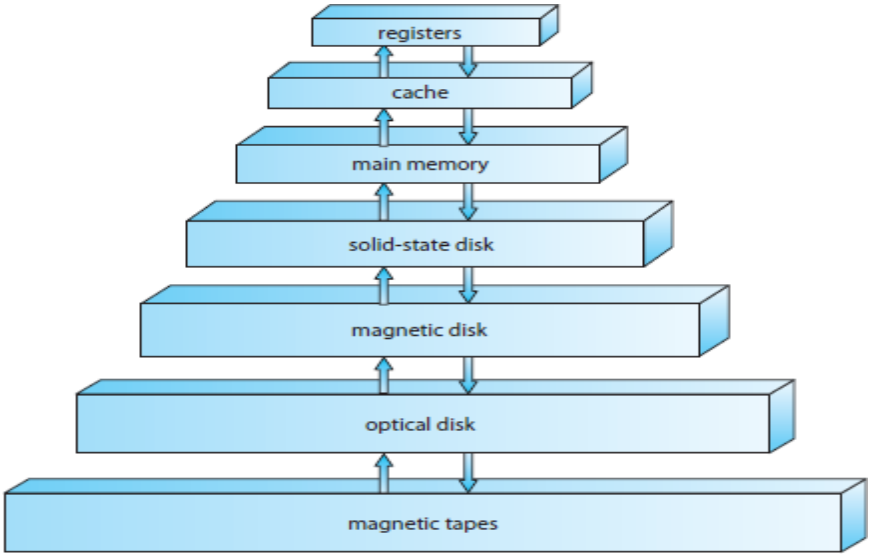
❖ Most computer systems provide **secondary storage** as an extension of main memory. The main requirement for secondary storage is that it be able to hold large quantities of data permanently.

❖ The wide variety of storage systems can be organized in a hierarchy according to speed and cost.

❖ The higher levels are expensive, but they are fast. As we move down the hierarchy, the cost per bit generally decreases, whereas the access time generally increases

❖ **Volatile storage** loses its contents when the power to the device is removed so that the data must be written to **nonvolatile storage** for safekeeping.

❖ Caches can be installed to improve performance where a large difference in access time or transfer rate exists between two components.



I/O Structure:

❖ A large portion of operating system code is dedicated to managing I/O, both because of its importance to the reliability and performance of a system.

❖ A general-purpose computer system consists of CPUs and multiple device controllers that are connected through a common bus. Each device controller is in charge of a specific type of device.

❖ The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage

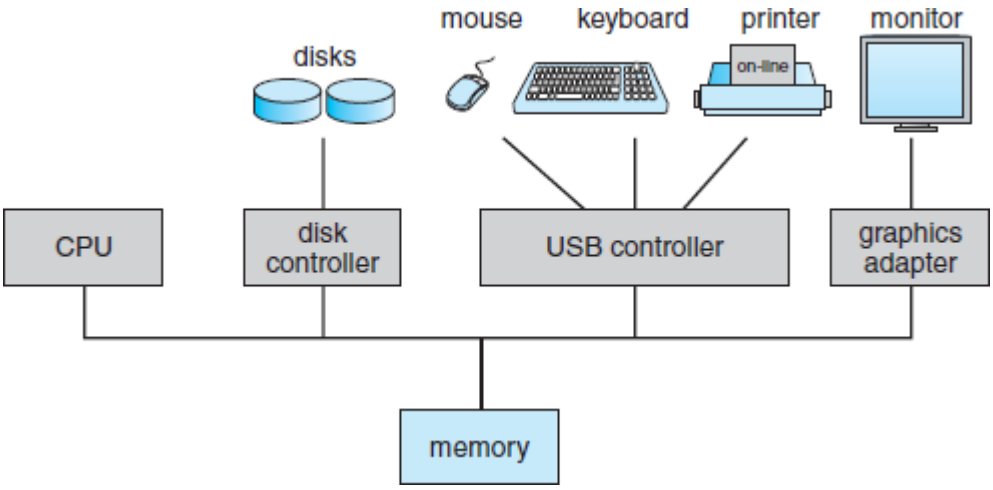
❖ Operating systems have a **device driver** for each device controller. This device driver understands the device controller and provides the rest of the operating system with a uniform interface to the device.

❖ To start an I/O operation, the device driver loads the appropriate registers within the device

controller.

❖ The controller starts the transfer of data from the device to its local buffer. Once the transfer of data is complete, the device controller informs the device driver via an interrupt that it has finished its operation. This is called as interrupt driven I/O.

The direct memory access I/O technique transfers a block of data directly to or from its own buffer storage to memory, with no intervention by the CPU. Only one interrupt is generated per block, to tell the device driver that the operation has completed,



2. FUNCTIONALITY OF OPERATING SYSTEM

POSSIBLE QUESTIONS:

1. With neat sketch discuss Computer System Overview (Nov 2015) (Apr 2022)

2. An OS is defined as a System program that controls the execution of application programs and acts as an interface between applications and the computer hardware.

OPERATING SYSTEM OBJECTIVES AND FUNCTIONS:

❖ An operating system is a program that manages a computer's hardware. It also provides a basis for application programs and acts as an intermediary between the computer user and the computer hardware. It can be thought of as having three objectives:

- **Convenience**
- **Efficiency**
- **Ability to evolve**

❖ The three other aspects of the operating system are

i) The operating system as a user or computer interface

ii) The operating system as a resource manager

iii) Ease of evolution of an operating system.

iv)

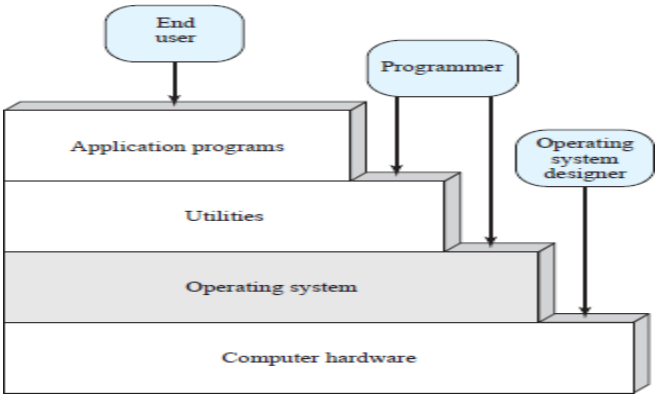
i) **The Operating System as a User/Computer Interface**

❖ The user of those applications, the end user, generally is not concerned with the details of computer hardware.

❖ An application can be expressed in a programming language and is developed by an application programmer.

❖ A set of system programs referred to as utilities implement frequently used functions that assist in program creation, the management of files, and the control of I/O devices.

❖ The most important collection of system programs comprises the OS. The OS masks the details of the hardware from the programmer and provides the programmer with a convenient interface for using the system.



❖ Briefly, the OS typically provides services in the following areas:

- Program development
- Program execution
- Access to I/O devices
- Controlled access to files
- System access
- Error detection and response
- Accounting:

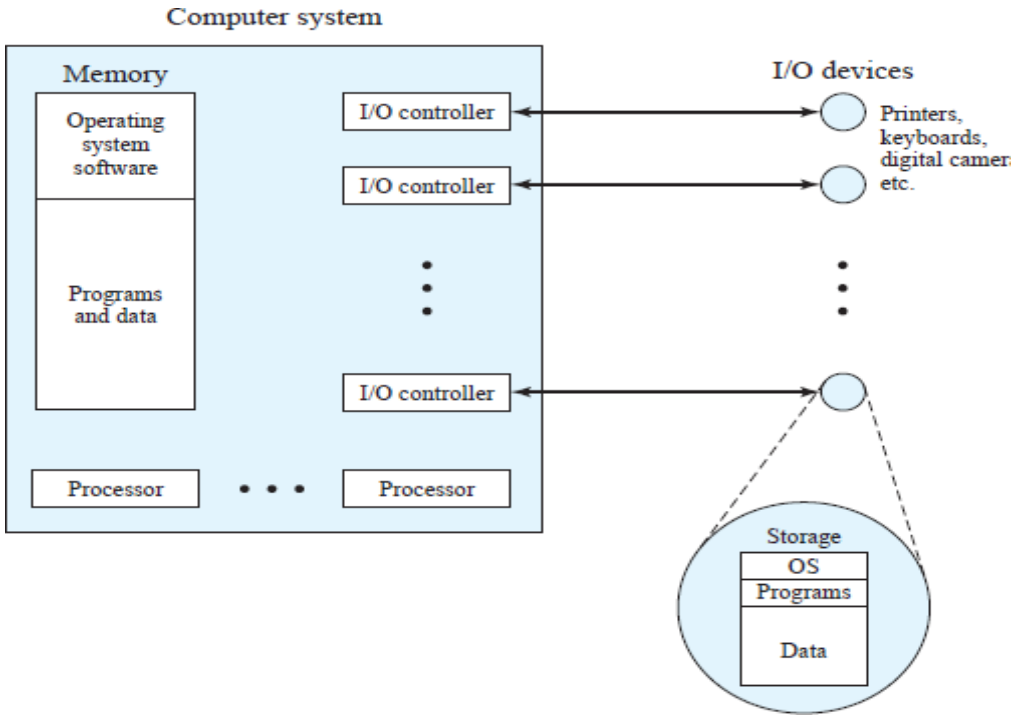
i) The Operating System as Resource Manager

❖ A computer is a set of resources for the movement, storage, and processing of data and for the control of these functions. The OS is responsible for managing these resources.

❖ The OS functions in the same way as ordinary computer software; that is, it is a program or suite of programs executed by the processor.

❖ The OS frequently relinquishes control and must depend on the processor to allow it to regain control.

❖ The OS directs the processor in the use of the other system resources and in the timing of its execution of other programs.



Operating system as a resource manager

❖ A portion of the OS is in main memory. This includes the kernel, or nucleus, which contains the most frequently, used functions in the OS.The remainder of main memory contains user programs and data.

❖ The allocation of this resource (main memory) is controlled jointly by the OS and memory management hardware in the processor.

❖ The OS decides when an I/O device can be used by a program in execution and controls access to and use of files.

❖ The processor itself is a resource, and the OS must determine how much processor time is to be devoted to the execution of a particular user program. In the case of a multiple-processor system, this decision must span all of the processors.

iii) Ease of Evolution of an Operating System

A major operating system will evolve over time for a number of reasons:

➢ **Hardware upgrades plus new types of hardware**

➢ **New services:** OS expands to offer new services in response to user demands.

➢ **Fixes**: Any OS has faults.

❖ The functions of operating system includes,

- Process management
- Memory management
- File management
- I/O management
- Storage management.

3. EVOLUTION OF OPERATING SYSTEM

❖ An operating system acts as an intermediary between the user of a computer and the computer hardware. The evolution of operating system is explained at various stages.

i) Serial Processing

ii) Simple Batch Systems

iii) Multiprogrammed batch systems.

iv) Time sharing systems

i) Serial processing

❖ During the 1940s to the mid-1950s, the programmer interacted directly with the computer hardware; there was no OS.

❖ Programs in machine code were loaded via the input device (e.g., a card reader).

❖ If an error halted the program, the error condition was indicated by the lights.

❖ If the program proceeded to a normal completion, the output appeared on the printer. These early systems presented two main problems:

i) **Scheduling**:

Most installations used a hardcopy sign-up sheet to reserve computer time. A user might sign up for an hour and finish in 45 minutes; this would result in wasted computer processing time. On the other hand, the user might run into problems, not finish in the allotted time, and be forced to stop before resolving the problem.

ii) **Setup time:** A single program, called a job, could involve loading the compiler plus the high-level language program (source program) into memory, saving the compiled program (object program) and then loading and linking together the object program and common functions. Thus, a considerable amount of time was spent just in setting up the program to run.

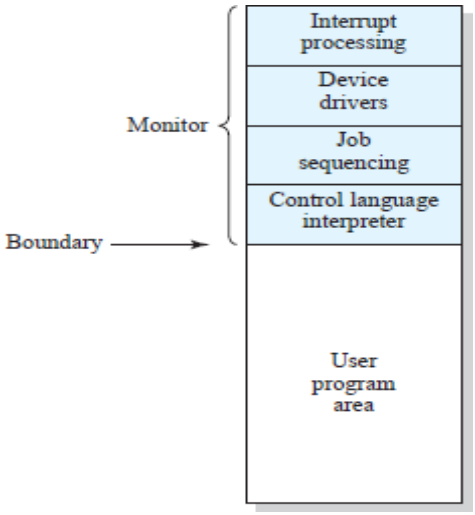
❖ This mode of operation could be termed serial processing, reflecting the fact that users have access to the computer in series

ii) Simple Batch Systems

❖ The central idea behind the simple batch-processing scheme is the use of a piece of software known as the **monitor.**

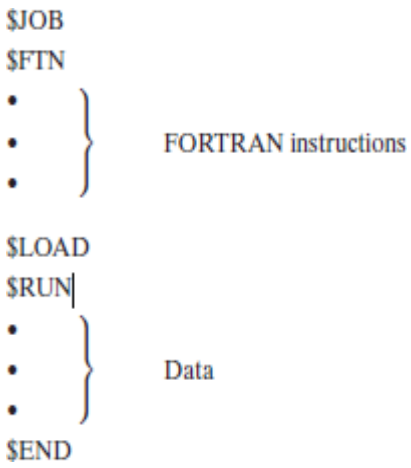
❖ With this type of OS, the user no longer has direct access to the processor. Instead, the user submits the job on cards or tape to a computer operator, who batches the jobs together sequentially and places the entire batch on an input device, for use by the monitor.

❖ Each program is constructed to branch back to the monitor when it completes processing, and the monitor automatically begins loading the next program.



Memory Layout for a Resident Monitor

- The monitor controls the sequence of events. For this the monitor must always be in main memory and available for execution. That portion is referred to as the resident monitor.
- The monitor reads in jobs one at a time from the input device .As it is read in, the current job is placed in the user program area, and control is passed to this job.
- Once a job has been read in, the processor will encounter a branch instruction in the monitor that instructs the processor to continue execution at the start of the user program. The processor will then execute the instructions in the user program until it encounters an ending or error condition.
- When the job is completed, it returns control to the monitor, which immediately reads in the next job. The results of each job are sent to an output device, such as a printer, for delivery to the user.
- The monitor performs a scheduling function: A batch of jobs is queued up, and jobs are executed as rapidly as possible, with no intervening idle time.
- With each job, instructions are included in a form of job control language (JCL) which are denoted by the beginning $. This is a special type of programming language used to provide instructions to the monitor.
- The overall format of the job is given as



❖ The hardware features that are added as a part of simple batch systems include,

i) Memory protection

ii) Timer

iii) Privileged instructions

iv) Interrupts.

❖ The memory protection leads to the concept of dual mode operation.

➢ User Mode

➢ Kernel Mode.

❖ Thus the simple batch system improves utilization of the computer

iii) Multiprogrammed Batch Systems:

❖ Even in simple batch operating system, the processor is often idle. The problem is that I/O devices are slow compared to the processor.

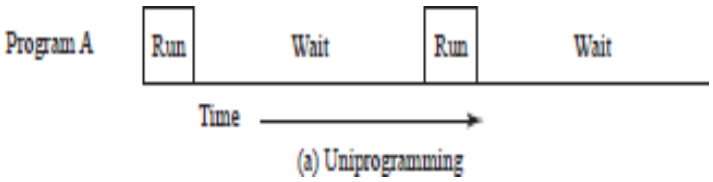
❖ Let us consider a program that processes a file of records and performs, on average, 100 machine instructions per record. The computer spends over 96% of its time waiting for I/O devices to finish transferring data to and from the file.

Read one record from file	15 μs
Execute 100 instructions	1 μs
Write one record to file	15 μs
Total	31 μs

$$\text{Percent CPU Utilization} = \frac{1}{31} = 0.032 = 3.2\%$$

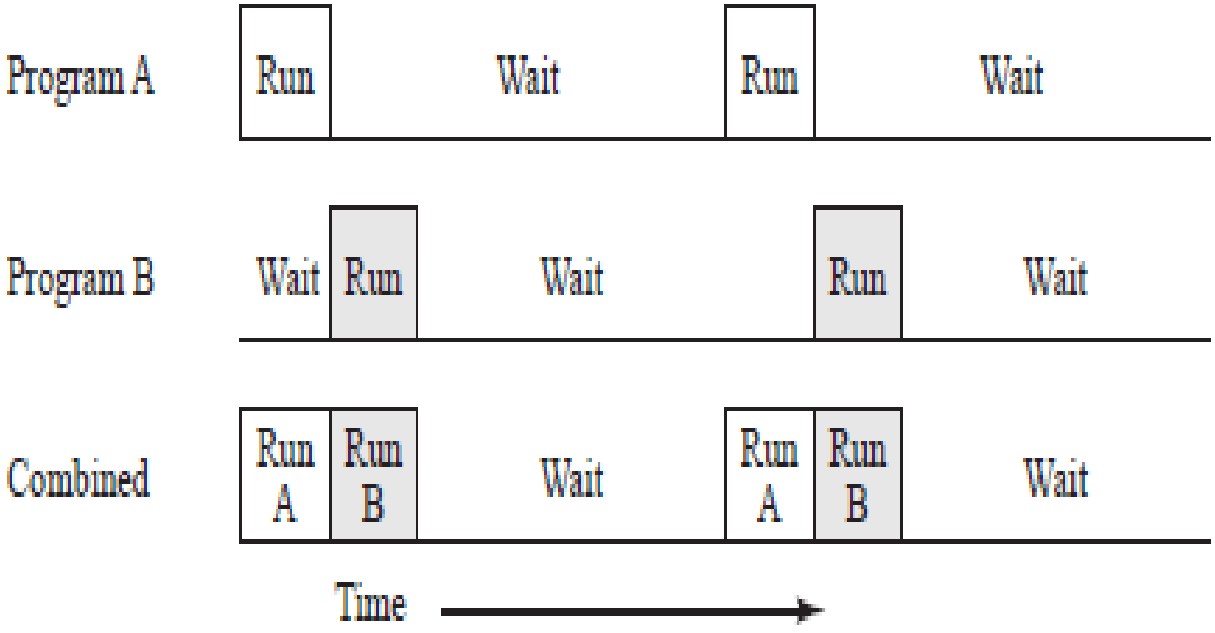
❖ In uniprogramming we will have a single program in the main memory. The processor spends a certain amount of time executing, until it reaches an I/O instruction. It must then wait until that I/O

instruction concludes before proceeding. This inefficiency is not necessary.

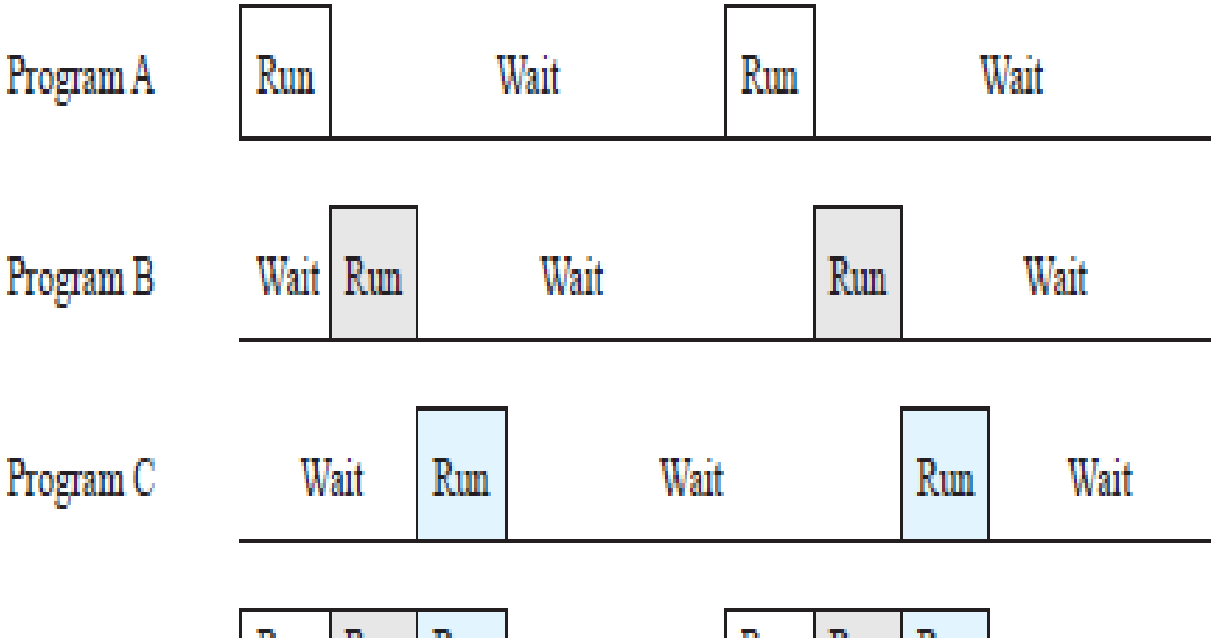


(a) Uniprogramming

❖ In Multiprogramming we will have OS and more user programs. When one job needs to wait for I/O, the processor can switch to the other job, which is likely not waiting for I/O.This approach is known as **multiprogramming, or multitasking**.



(b) Multiprogramming with two programs



(c) Multiprogramming with three programs

- The most notable feature that is useful for multiprogramming is the hardware that supports I/O interrupts and DMA (direct memory access).
- With interrupt-driven I/O or DMA, the processor can issue an I/O command for one job and proceed with the execution of another job while the I/O is carried out by the device controller.
- When the I/O operation is complete, the processor is interrupted and control is passed to an interrupt-handling program in the OS.The OS will then passes control to another job.
- Multiprogramming operating systems are fairly sophisticated compared to single-program, or uniprogramming, systems. To have several jobs ready to run, they must be kept in main memory, requiring some form of memory management.
- In addition, if several jobs are ready to run, the processor must decide which one to run, this decision requires an algorithm for scheduling.

iv) Time-Sharing Systems:

POSSIBLE QUESTIONS:

1. Under what circumstances would a user be better off using a timesharing system rather than a PC or single -user workstation.(Nov 2018)

- In time sharing systems the processor time is shared among multiple users.
- In a time-sharing system, multiple users simultaneously access the system through terminals, with the OS interleaving the execution of each user program in a short burst or quantum of computation.
- If there are n users actively requesting service at one time, each user will only see on the average 1/n of the effective computer capacity.
- **Batch Multiprogramming Vs Time Sharing systems**

	Batch Multiprogramming	Time Sharing
Principal objective	Maximize processor use	Minimize response time
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

- One of the first time-sharing operating systems to be developed was the Compatible Time-Sharing System (CTSS)
- The system ran on a computer with 32,000 36-bit words of main memory, with the resident monitor consuming 5000 of that. When control was to be assigned to an interactive user, the user's program and data were loaded into the remaining 27,000 words of main memory.
- A program was always loaded to start at the location of the 5000th word
- A system clock generated interrupts at a rate of approximately one every 0.2 seconds.
- At each clock interrupt, the OS regained control and could assign the processor to another user.

This technique is known as **time slicing.**

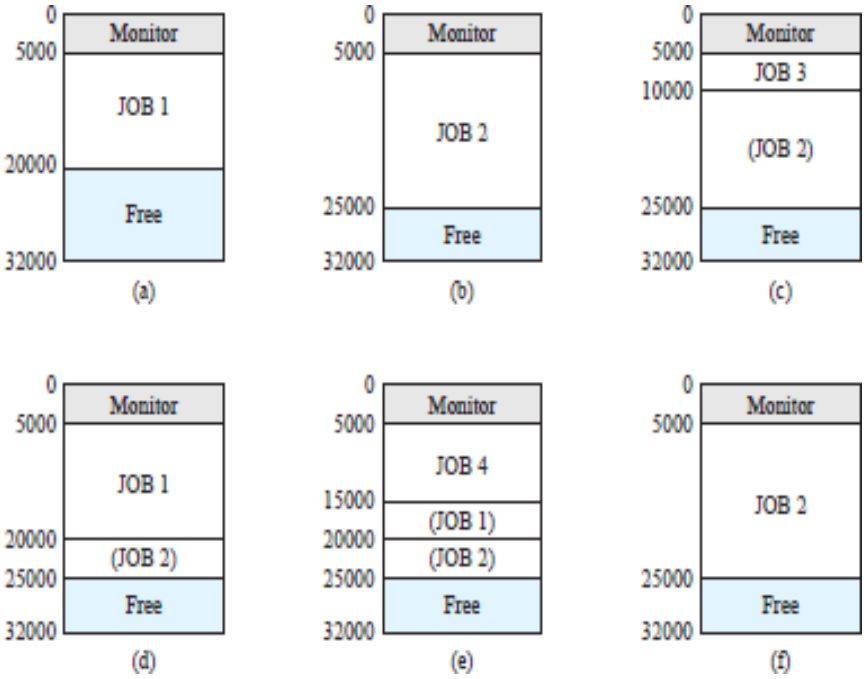
Example: Assume that there are four interactive users with the following memory requirements, in words:

JOB1: 15,000

JOB2: 20,000

JOB3: 5000

JOB4: 10,000



i) Initially, the monitor loads JOB1 and transfers control to it.

ii) Later, the monitor decides to transfer control to JOB2. Because JOB2 requires more memory than JOB1, JOB1 must be written out first, and then JOB2 can be loaded.

iii) Next, JOB3 is loaded in to be run. However, because JOB3 is smaller than JOB2, a portion of JOB2 can remain in memory, reducing disk write time.

iv) Later, the monitor decides to transfer control back to JOB1.An additional portion of JOB2 must be written out when JOB1 is loaded back into memory.

v) When JOB4 is loaded, part of JOB1 and the portion of JOB2 remaining in memory are retained.

vi) At this point, if either JOB1 or JOB2 is activated, only a partial load will be required. In this example, it

is JOB2 that runs next. This requires that JOB4 and the remaining resident portion of JOB1 be written out and that the missing portion of JOB2 be read in.

4. OPERATING SYSTEM STRUCTURING METHODS

POSSIBLE QUESTIONS:

1. Enumerate different operating system structures and explain with neat sketch (April2017) (May 2019)

2. What is dual mode operation and what is the need of it? (May 2019) (Apr 2022)

❖ The operating systems are large and complex. A common approach is to partition the task into small components, or modules, rather than have one **monolithic** system.

❖ The structure of an operating system can be defined the following structures.

- Monolithic
- Layered approach
- Microkernels
- Modules
- Hybrid systems

Monolithic structure:

The operating system (OS) is written as a collection of procedures, each of which can call any of the other ones, whenever it needs to.

Monolithic systems provides a basic structure for the operating system .

a main program that invokes the requested service procedure

a set of service procedures that carry out the system calls

a set of utility procedures that help the service procedure

In Monolithic system model, there is one service procedure for each system call, that takes care of it.

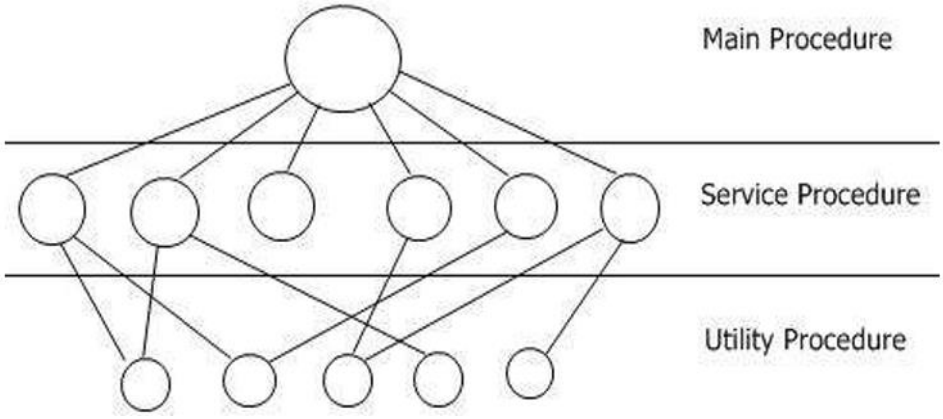
The utility procedures do jobs that are needed by several service procedures, such as fetching the data from the user programs.

This division of the procedures into the following three layers:

Main Procedure

Service Procedures

Utility Procedures

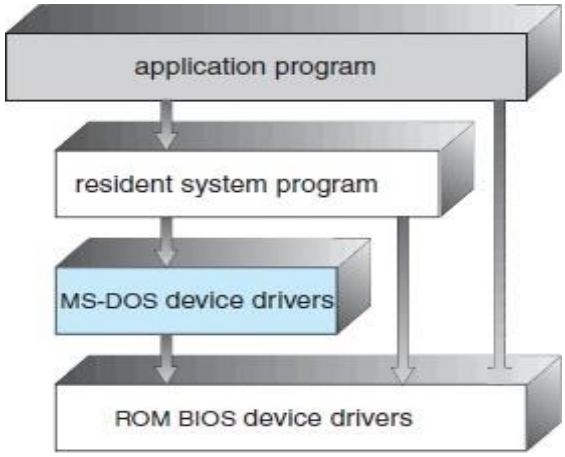


❖ The Simple structured operating systems do not have a well defined structure. These systems will be simple, small and limited systems.

Example: MS-DOS.

❖ In MS-DOS, the interfaces and levels of functionality are not well separated.

❖ In MS-DOS application programs are able to access the basic I/O routines. This causes the entire systems to be crashed when user programs fail.



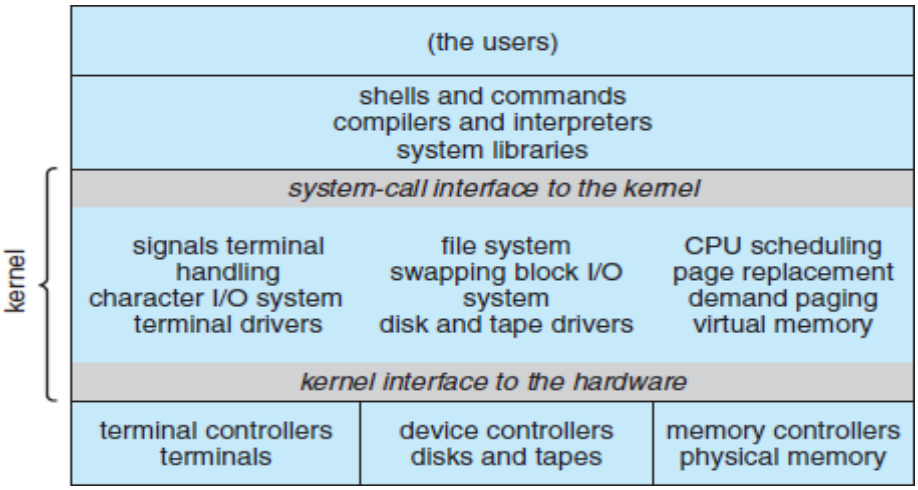
Example: Traditional UNIX OS

❖ It consists of two separable parts: the kernel and the system programs.

❖ The kernel is further separated into a series of interfaces and device drivers

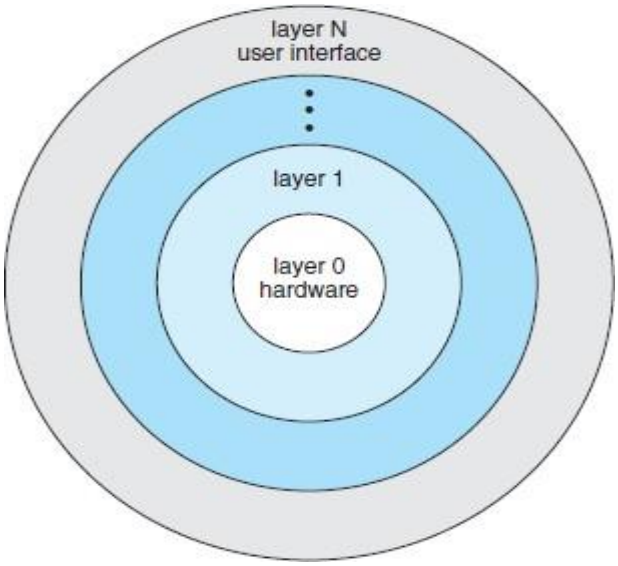
❖ The kernel provides the file system, CPU scheduling, memory management, and other operating-system functions through system calls.

❖ This monolithic structure was difficult to implement and maintain.



Layered approach:

❖ A system can be made modular in many ways. One method is the **layered approach**, in which the operating system is broken into a number of layers (levels). The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.



❖ An operating-system layer is an implementation of an abstract object made up of data and the operations that can manipulate those data.

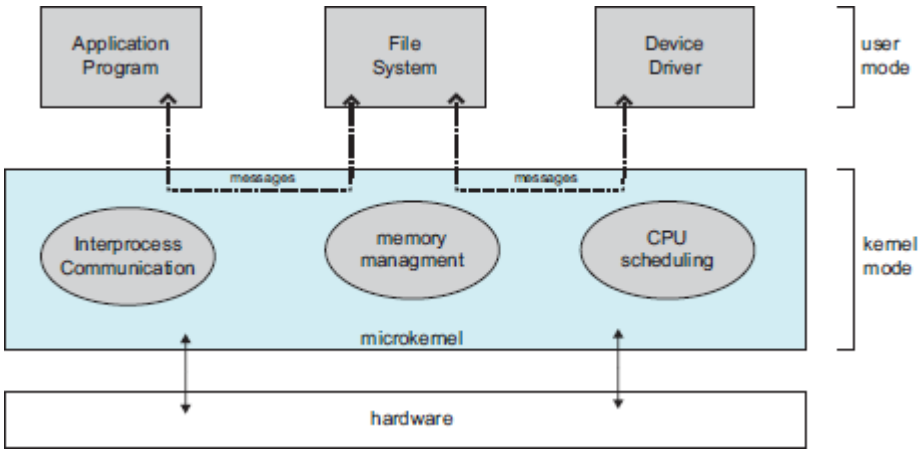
❖ The main advantage of the layered approach is simplicity of construction and debugging. The layers are selected so that each uses functions (operations) and services of only lower-level layers.

❖ Each layer is implemented only with operations provided by lower-level layers. A layer does not need to know how these operations are implemented; it needs to know only what these operations do.

- The major difficulty with the layered approach involves appropriately defining the various layers because a layer can use only lower-level layers.
- A problem with layered implementations is that they tend to be less efficient than other types.

Microkernels:

- In the mid-1980s, researchers at Carnegie Mellon University developed an operating system called **Mach** that modularized the kernel using the **microkernel** approach.
- This method structures the operating system by removing all nonessential components from the kernel and implementing them as system and user-level programs.



- Microkernel provide minimal process and memory management, in addition to a communication facility.
- The main function of the microkernel is to provide communication between the client program and the various services that are also running in user space.
- The client program and service never interact directly. Rather, they communicate indirectly by exchanging messages with the microkernel.
- One benefit of the microkernel approach is that it makes extending the operating system easier. All new services are added to user space and consequently do not require modification of the kernel.
- The performance of microkernel can suffer due to increased system-function overhead.

Microkernel and system applications can interact with each other by message passing as and when required. This is extremely advantageous architecture since burden of kernel is reduced and less crucial services are accessible to the user and hence security is improved too. It is being highly adopted in the present-day systems.

Advantages:

Kernel is small and isolated and can hence function better

Expansion of the system is easier, it is simply added in the system application without disturbing the kernel.

Benefits:

Easier to extend a microkernel

Easier to port the operating system to new architectures

More reliable (less code is running in kernel mode)

More secure

Modules:

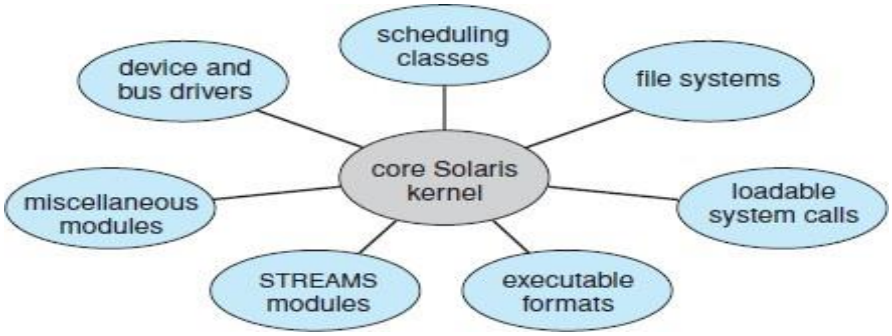
❖ The best current methodology for operating-system design involves using **loadable kernel modules**

❖ The kernel has a set of core components and links in additional services via modules, either at boot time or during run time.

❖ The kernel provides core services while other services are implemented dynamically, as the kernel is running.

❖ Linking services dynamically is more comfortable than adding new features directly to the kernel, which would require recompiling the kernel every time a change was made.

Example: Solaris OS



❖ The Solaris operating system structure is organized around a core kernel with seven types of loadable kernel modules:

- Scheduling classes
- File systems
- Loadable system calls
- Executable formats
- STREAMS modules
- Miscellaneous
- Device and bus drivers

Hybrid Systems:

❖ The Operating System combines different structures, resulting in hybrid systems that address performance, security, and usability issues.

❖ They are monolithic, because having the operating system in a single address space provides

very efficient performance.

❖ However, they are also modular, so that new functionality can be dynamically added to the kernel.

❖ Example: Linux and Solaris are monolithic (simple) and also modular, IOS.

❖ Apple IOS Structure

● The main function of the command interpreter is to get and execute the next user-specified command. Many of the commands given at this level manipulate files: create, delete, list, print, copy, execute, and so on. The MS-DOS and UNIX shells operate in this way.

● These commands can be implemented in two general ways. In one approach, the command interpreter itself contains the code to execute the command. For example, a command to delete a file may cause the command interpreter to jump to a section of its code that sets up the parameters and makes the appropriate system call. In this case, the number of commands that can be given determines the size of the command interpreter, since each command requires its own implementing code. An alternative approach—used by UNIX, among other operating systems —implements most commands through system programs.

● In this case, the command interpreter does not understand the command in any way; it merely uses the command to identify a file to be loaded into memory and executed. Thus, the UNIX command to delete a file rm file.txt would search for a file called rm, load the file into memory, and execute it with the parameter file.txt.

5. SYSTEM CALLS

POSSIBLE QUESTIONS:

1. Define system call? What is the purpose of system calls? (April 2018)

2. Why API's need to be used rather than system calls? (April 2015)

3. List and explain the classification of system calls and its significance (Nov 2018)

4. Describe System Calls, System Programs in detail with neat sketch. (April 2017, Dec 2021)

❖ The system call provides an interface to the operating system services.

❖ Application developers often do not have direct access to the system calls, but can access them through an application programming interface (API). The functions that are included in the API invoke the actual system calls.

❖ Systems execute thousands of system calls per second. Application developers design programs according to an **application programming interface (API)**.

❖ For most programming languages, the Application Program Interface provides a **system call**

interface that serves as the link to system calls made available by the operating system.

❖ The system-call interface intercepts function calls in the API and invokes the necessary system calls within the Operating system.

❖ **Example: System calls for writing a simple program to read data from one file and copy them to another file**

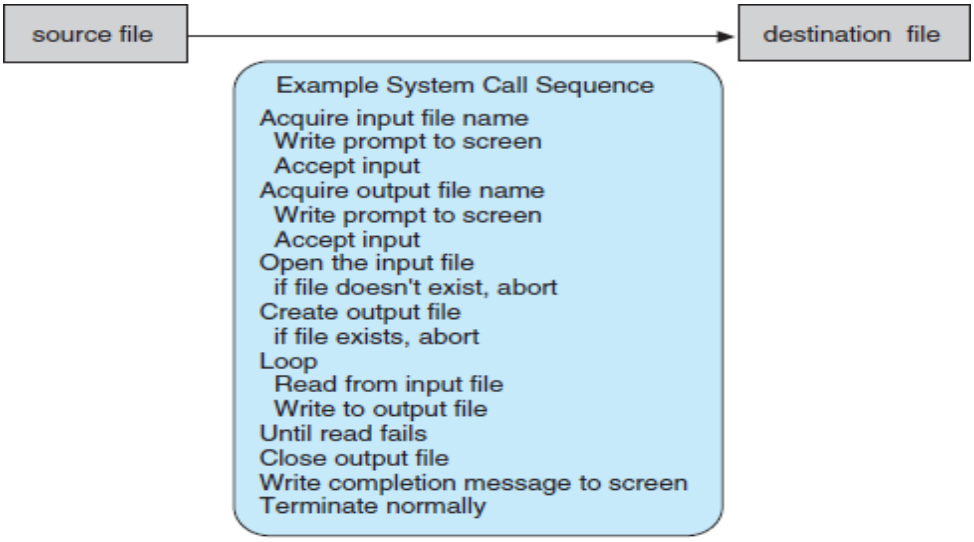


Figure 2.5 Example of how system calls are used.

❖ The caller of the system call need know nothing about how the system call is implemented or what it does during execution.

❖ The caller need only obey the API and understand what the operating system will do as a result of the execution of that system call.

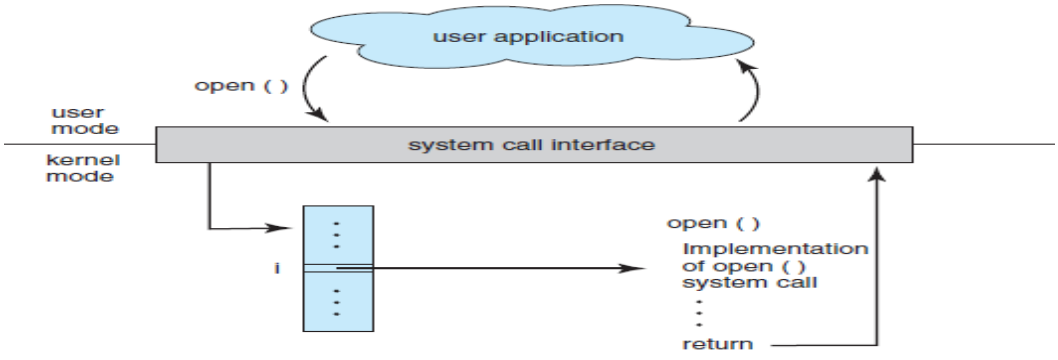


Figure 2.6 The handling of a user application invoking the open ( ) system call.

❖ Three general methods are used to pass parameters to the operating system

○ pass the parameters in registers

○ parameters are generally stored in a block, or table, in memory, and the address of the block is passed as a parameter in a register

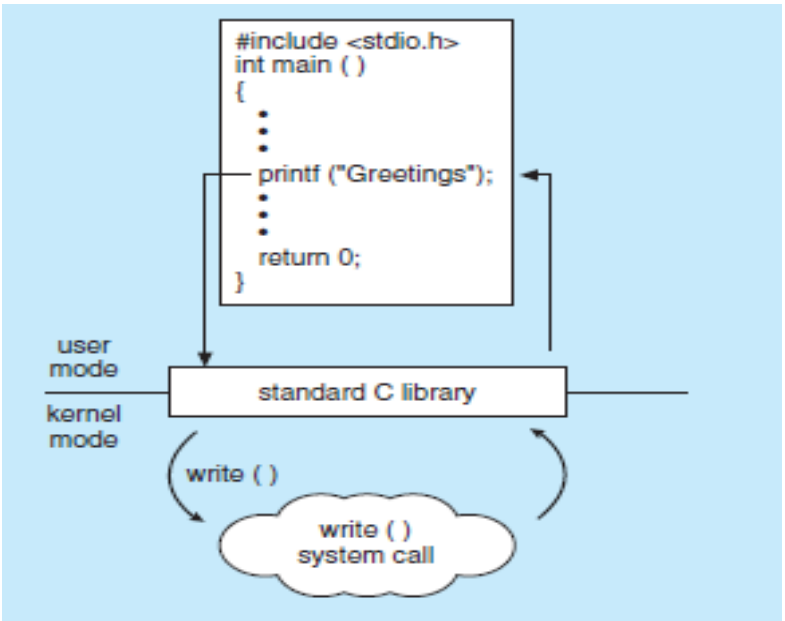
- Parameters also can be placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system.

Types of System Calls:

- System calls can be grouped roughly into six major categories
  - Process control,
  - File manipulation,
  - Device manipulation,
  - Information maintenance,
  - Communications,
  - Protection.

PROCESS CONTROL:

- A Running program needs to be able to halt its execution either normally (end ()) or abnormally (abort()).
- Under either normal or abnormal circumstances, the operating system must transfer control to the invoking command interpreter.
- A process or job executing one program may want to load() and execute() another program. This feature allows the command interpreter to execute a program as directed by, for example, a user command, the click of a mouse, or a batch command.



- If we create a new job or process, or perhaps even a set of jobs or processes, we should be able to control its execution that requires to determine and reset the attributes of a job or process, including the job's priority, its maximum allowable execution time, and so on (get process attributes() and set process attributes()).
- We may also want to terminate a job or process that we created (terminate process()) if we find that it is incorrect or is no longer needed.

❖ The System calls associated with process control includes

◦ **end, abort**

◦ **load, execute**

◦ **create process, terminate process**

◦ **get process attributes, set process attributes**

◦ **Wait for time**

◦ **wait event, signal event**

◦ **allocate and free memory**

❖ When a process has been created We may want to wait for a certain amount of time to pass (wait time()) or we will want to wait for a specific event to occur (wait event()).

❖ The jobs or processes should then signal when that event has occurred (signal event()

❖ To start a new process, the shell executes a fork() system call. Then, the selected program is loaded into memory via an exec() system call, and the program is executed

❖ When the process is done, it executes an exit() system call to terminate, returning to the invoking process a status code of 0 or a nonzero error code.

FILE MANAGEMENT:

❖ In order to work with files We first need to be able to create () and delete () files. Either system call requires the name of the file and perhaps some of the file's attributes. Once the file is created, we need to open() it and to use it.

❖ We may also read (), write (), or reposition ().Finally, we need to close () the file, indicating that we are no longer using it.

❖ In addition, for either files or directories, we need to be able to determine the values of various attributes and perhaps to reset them if necessary.

❖ File attributes include the file name, file type, protection codes, accounting information, and so on. At least two system calls, get file attributes () and set file attributes (), are required for this function.

❖ The System calls associated with File management includes

○ File management

○ create file, delete file

○ open, close

○ read, write, reposition

○ get file attributes, set file attributes

DEVICE MANAGEMENT:

❖ A process may need several resources to execute—main memory, disk drives, access to files, and so on. If the resources are available, they can be granted, and control can be returned to the user process.

Otherwise, the process will have to wait until sufficient resources are available.

❖ A system with multiple users may require us to first request() a device, to ensure exclusive use of it.

❖ After we are finished with the device, we release() it. These functions are similar to the open() and close() system calls for files.

Once the device has been requested (and allocated to us), we can read(), write(), and (possibly) reposition() the device, just as we can with files.

❖ I/O devices are identified by special file names, directory placement, or file attributes.

❖ The System calls associated with Device management includes

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

INFORMATION MAINTENANCE:

❖ Many system calls exist simply for the purpose of transferring information between the user program and the operating system.

❖ Example, most systems have a system call to return the current time() and date().

❖ Other system calls may return information about the system, such as the number of current users, the version number of the operating system, the amount of free memory or disk space, and so on.

❖ Many systems provide system calls to dump() memory. This provision is useful for debugging.

❖ Many operating systems provide a time profile of a program to indicate the amount of time that the program executes at a particular location or set of locations.

❖ The operating system keeps information about all its processes, and system calls are used to access this information.

❖ Generally, calls are also used to reset the process information (get process attributes() and set process attributes()).

❖ The System calls associated with Device management includes

- get time or date, set time or date
- get system data, set system data
- get process, file, or device attributes
- set process, file, or device attributes

COMMUNICATION:

❖ There are two common models of Interprocess communication: the message passing model and the shared-memory model.

❖ In the **message-passing model**, the communicating processes exchange messages with one another to transfer information.

❖ Messages can be exchanged between the processes either directly or indirectly through a common mailbox.

❖ Each process has a **process name**, and this name is translated into an identifier by which the operating system can refer to the process. The get hostid() and get processid() system calls do this translation.

❖ The recipient process usually must give its permission for communication to take place with an accept connection () call.

❖ The source of the communication, known as the **client**, and the receiving daemon, known as a **server,** then exchange messages by using read message() and write message() system calls.

❖ The close connection() call terminates the communication

❖ In the **shared-memory model**, processes use shared memory create() and shared memory attach() system calls to create and gain access to regions of memory owned by other processes.

❖ The system calls associated with communication includes,

◦ create, delete communication connection

◦ send, receive messages

◦ Transfer status information

◦ attach or detach remote devices

PROTECTION:

❖ Protection provides a mechanism for controlling access to the resources provided by a computer system.

❖ System calls providing protection include set permission () and get permission (), which manipulate the permission settings of resources such as files and disks.

❖ The allow user () and deny user () system calls specify whether particular users can—or cannot—be allowed access to certain resources.

6. SYSTEM PROGRAMS

POSSIBLE QUESTIONS:

1. What is the purpose of system programs? (Apr 2022)

❖ **System programs**, also known as **system utilities**, provide a convenient environment for program development and execution.

❖ They can be divided into these categories:

○ File management

○ Status information

○ File modification.

○ Programming-language support

○ Program loading and execution

- Communications
- Background services

i) File Management:

- These programs create, delete, copy, rename, print, dump, list, and generally manipulate files and directories.

ii) **Status Information**:

- Some programs simply ask the system for the date, time, amount of available memory or disk space, number of users, or similar status information.
- Others are more complex, providing detailed performance, logging, and debugging information.

iii) File Modification:

- Several text editors may be available to create and modify the content of files stored on disk or other storage devices
- There may also be special commands to search contents of files or perform transformations of the text.

iv) Programming Language support:

- Compilers, assemblers, debuggers, and interpreters for common programming languages (such as C, C++, Java, and PERL) are often provided with the operating system.

v) Program Loading and Execution:

- Once a program is assembled or compiled, it must be loaded into memory to be executed.
- The system may provide absolute loaders, relocatable loader.

vi)**Communication:**

- These programs provide the mechanism for creating virtual connections among processes, users,
- and computer systems.
- They allow users to send messages to one another's screens, to browse Web pages, to send e-mail messages, to log in remotely, or to transfer files from one machine to another.

vii) Background Services:

- All general-purpose systems have methods for launching certain system-program processes at boot time.
- Some of these processes terminate after completing their tasks, while others continue to run until the system is halted. Constantly running system-program processes are known as **services**, **subsystems**, or daemons.
- Along with system programs, most operating systems are supplied with programs that are useful in solving common problems or performing common operations.
- Such **application programs** include Web browsers, word processors and text formatters, spreadsheets, database systems, compilers, plotting and statistical-analysis packages, and games.

7. HARDWARE PROTECTION

Operating-System Operations

1. Modern operating systems are interrupt driven. If there are no processes to execute, no I/O devices to service, and no users to whom to respond, an operating system will sit quietly, waiting for something to happen. Events are almost always signaled by the occurrence of an interrupt or a trap.
2. A trap (or an exception) is a software-generated interrupt caused either by an error or by a specific request from a user program that an operating-system service be performed.
3. The interrupt-driven nature of an operating system defines that system's general structure. For each type of interrupt, separate segments of code in the operating system determine what action should be taken. An interrupt service routine is provided that is responsible for dealing with the interrupt.
4. The operating system and the users share the hardware and software resources of the computer system, we need to make sure that an error in a user program could cause problems only for the one program that was running. With sharing, many processes could be adversely affected by a bug in one program. For example, if a process gets stuck in an infinite loop, this loop could prevent the correct operation of many other processes.
5. Without protection against these sorts of errors, either the computer must execute only one process at a time or all output must be suspect.

Dual-Mode Operation

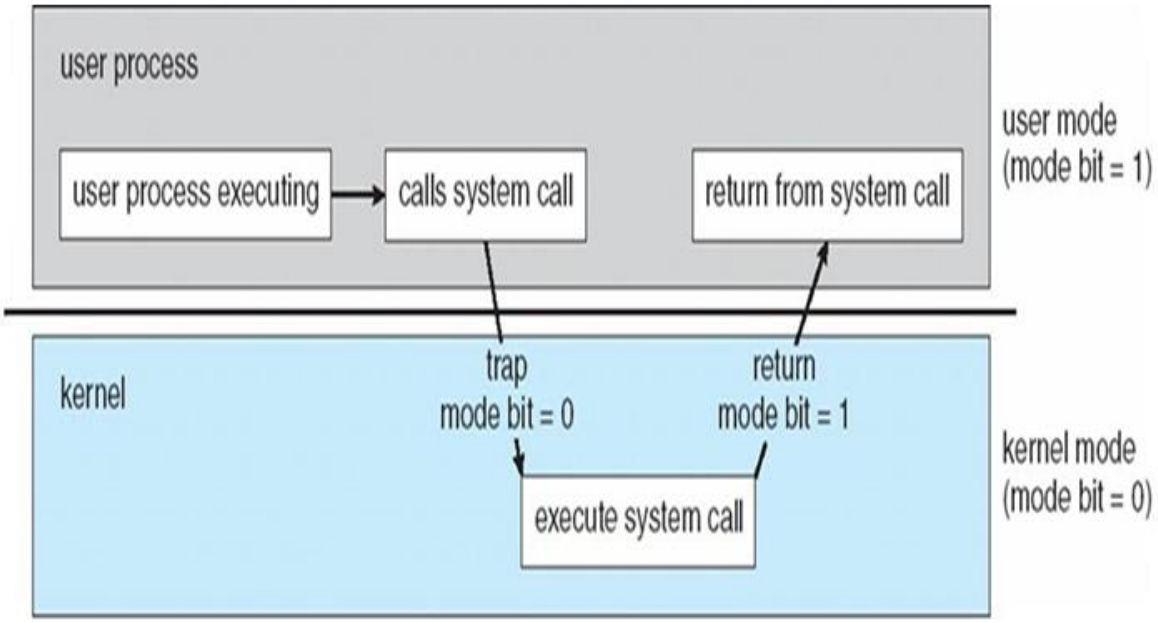
Dual-mode operation allows OS to protect itself and other system components

User mode and kernel mode

Mode bit provided by hardware Provides ability to distinguish when system is running user code or kernel code Some instructions designated as privileged, only executable in kernel mode System call changes mode to kernel, return from call resets it to user.

Transition from User to Kernel Mode

- Timer to prevent infinite loop / process hogging resources Set interrupt after specific period
- Operating system decrements counter
- When counter zero generate an interrupt
- Set up before scheduling process to regain control or terminate program that exceeds allotted time



Protection and Security

If a computer system has multiple users and allows the concurrent execution of multiple processes, then access to data must be regulated. For that purpose, mechanisms ensure that files, memory segments, CPU, and other resources can be operated on by only those processes that have gained proper authorization from the operating system.

Protection is any mechanism for controlling the access of processes or users to the resources

defined by a computer system. This mechanism must provide means for specification of the controls to be imposed and means for enforcement.

Protection can improve reliability by detecting latent errors at the interfaces between component subsystems. Early detection of interface errors can often prevent contamination of a healthy subsystem by another subsystem that is malfunctioning. An unprotected resource cannot defend against use (or misuse) by an unauthorized or incompetent user. A protection-oriented system provides a means to distinguish between authorized and unauthorized usage, A system can have adequate protection but still be prone to failure and allow inappropriate access.

It is the job of security to defend a system from external and internal attacks. Such attacks spread across a huge range and include viruses and worms, denial-of service attacks Protection and security require the system to be able to distinguish among all its users. Most operating systems maintain a list of user names and associated user identifiers (user IDs).

• User ID then associated with all files, processes of that user to determine access control

• Group identifier (group ID) allows set of users to be defined and controls managed, then also associated with each process, file Privilege escalation allows user to change to effective ID with more rights

Kernel Data Structures

The operating system must keep a lot of information about the current state of the system.

As things happen within the system these data structures must be changed to reflect the current reality.

For example, a new process might be created when a user logs onto the system. The kernel must create a data structure representing the new process and link it with the data structures representing all of the other processes in the system.

Mostly these data structures exist in physical memory and are accessible only by the kernel and its subsystems. Data structures contain data and pointers, addresses of other data structures, or the addresses of routines.

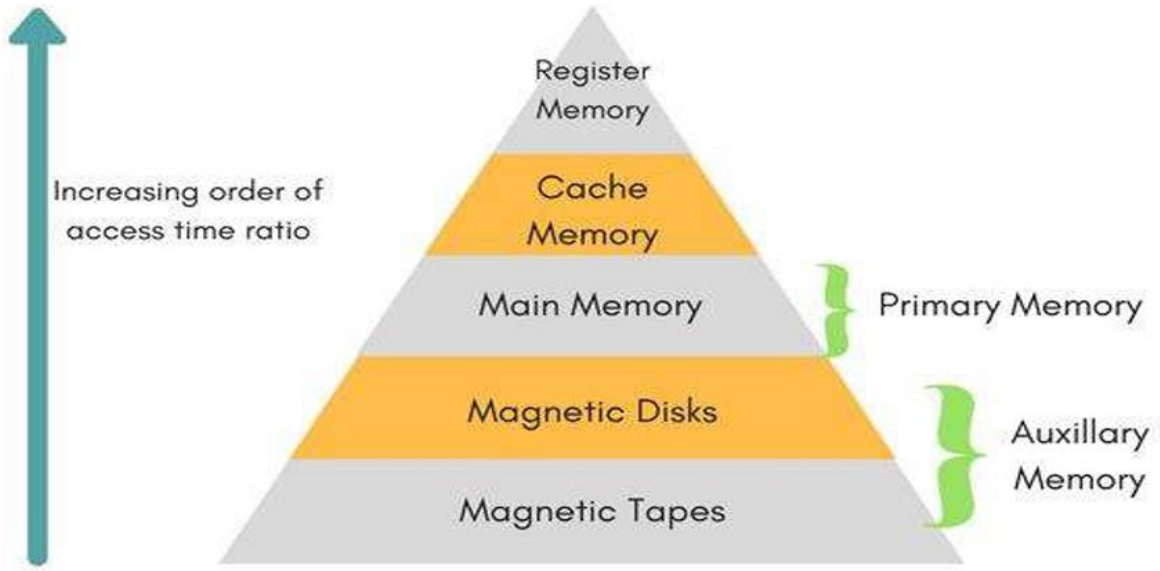
Taken all together, the data structures used by the Linux kernel can look very confusing.

Every data structure has a purpose and although some are used by several kernel subsystems, they are more simple than they appear at first sight.

Understanding the Linux kernel hinges on understanding its data structures and the use that the various functions within the Linux kernel makes of them. This section bases its description of the Linux kernel on its data structures. It talks about each kernel subsystem in terms of its algorithms, which are its methods of getting things done, and their usage of the kernel's data structures.

8. MEMORY HIERARCHY

A memory unit is the collection of storage units or devices together. The memory unit stores the binary information in the form of bits. Generally, memory/storage is classified into 2 categories: Volatile Memory: This loses its data, when power is switched off. Non-Volatile Memory: This is a permanent storage and does not lose any data when power is switched off.



Memory Hierarchy

Registers

Registers of the CPU are also storage devices. They hold data temporarily when CPU executes the instruction.

Cache Memory

Cache Memory is directly connected to the CPU as well as the main memory. Normally, Cache Memory receives the data from main memory. The Secondary Memory is connected to the main memory. The CPU does not read data from or write data to secondary memory.

Main Memory

In our Computer System is used to store data temporarily.

Secondary Memory

In our Computer System secondary memory is used to store data permanently.

The memory is characterized on the base of two factors:

Capacity

Capacity is the amount of information or data that a memory can store.

Access Time

Access time is the time interval between the read and write request and the availability of data

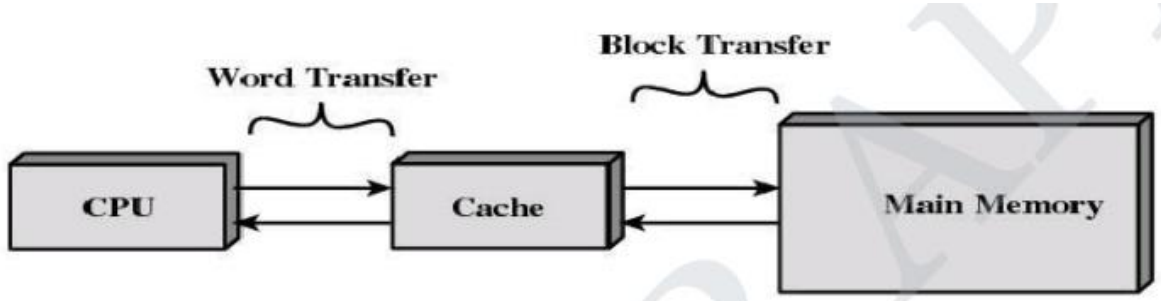
9. CACHE MEMORY

Cache (pronounced cash) memory is extremely fast memory that is built into a computer's central processing unit (CPU), or located next to it on a separate chip. The CPU uses cache memory to store instructions that are repeatedly required to run programs, improving overall system speed. Cache memory is invisible to the OS; it interacts with other memory management hardware.

Cache memory, also called CPU memory, is high-speed static random access memory (SRAM) that a computer microprocessor can access more quickly than it can access regular random access memory (RAM). Cache memory is an extremely fast memory type that acts as a buffer between RAM and the CPU.

It holds frequently requested data and instructions so that they are immediately available to the CPU when needed.

Cache memory is used to reduce the average time to access data from the Main memory. The cache is a smaller and faster memory which stores copies of the data from frequently used main memory locations. There are various different independent caches in a CPU, which stored instruction and data.



Cache Design

They fall into the following categories:

- **Cache size** - small caches can have a significant impact on performance
- **Block size** - the unit of data exchanged between cache and main memory.
- **Mapping function** - When a new block of data is read into the cache, the mapping function determines which cache location the block will occupy.
- **Replacement algorithm** - It chooses which block to replace when a new block is to be loaded into the cache and the cache already has all slots filled with other blocks. We would like to replace the block that is least likely to be needed again in the near future.
- **Write policy** - The write policy dictates when the memory write operation takes place. At one extreme, the writing can occur every time that the block is updated. At the other extreme, the writing occurs only when the block is replaced.

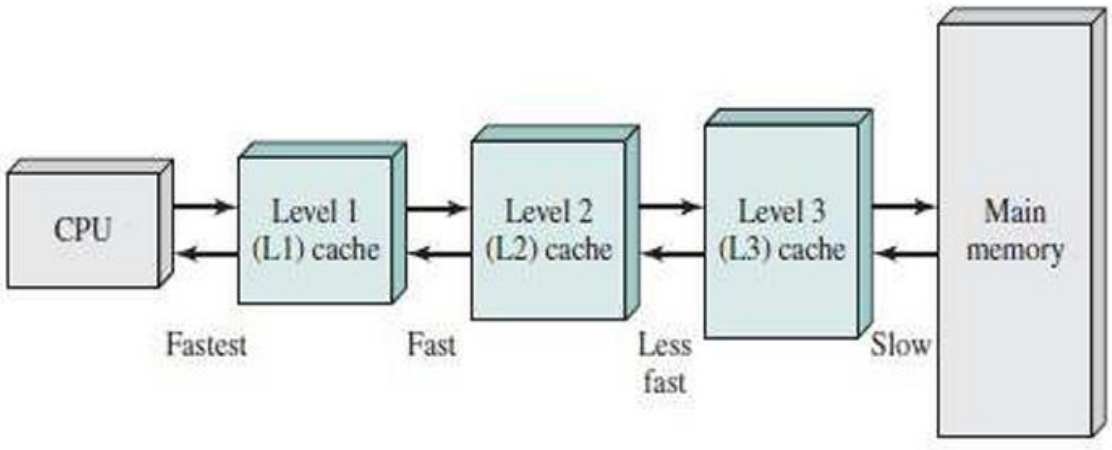
Levels of memory:

Level 1 or Register – It is a type of memory in which data is stored and accepted that are immediately stored in CPU. Most commonly used register is accumulator, Program counter, address register etc.

Level 2 or Cache memory – It is the fastest memory which has faster access time where data is temporarily stored for faster access.

Level 3 or Main Memory – It is memory on which computer works currently it is small in size and once power is off data no longer stays in this memory

Level 4 or Secondary Memory – It is external memory which is not fast as main memory but data stays permanently in this memory.



(b) Three-level cache organization

Hit Ratio

The performance of cache memory is measured in terms of a quantity called hit ratio. When the CPU refers to memory and finds the word in cache it is said to produce a hit. If the word is not found in cache, it is in main memory then it counts as a miss.

The ratio of the number of hits to the total CPU references to memory is called hit ratio.

Hit Ratio = Hit / (Hit + Miss) Cache memory mapping

Caching configurations continue to evolve, but cache memory traditionally works under three different configurations:

Direct mapped cache has each block mapped to exactly one cache memory location. Conceptually, direct mapped cache is like rows in a table with three columns: the data block or cache line that contains the actual data fetched and stored, a tag with all or part of the address of the data that was fetched, and a flag bit that shows the presence in the row entry of a valid bit of data.

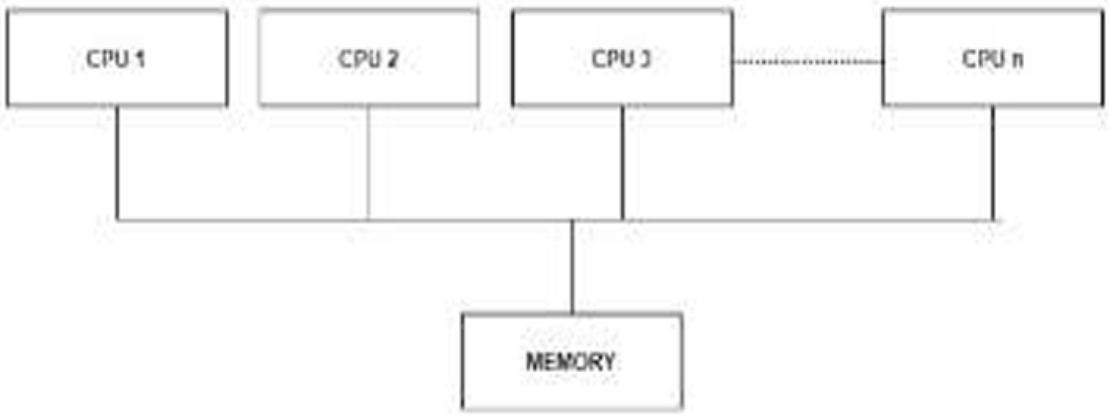
Fully associative cache mapping is similar to direct mapping in structure but allows a block to be mapped to any cache location rather than to a prespecified cache memory location as is the case with direct mapping.

Set associative cache mapping can be viewed as a compromise between direct mapping and fully associative mapping in which each block is mapped to a subset of cache locations. It is sometimes called N-way set associative mapping, which provides for a location in main memory to be cached to any of "N" locations in the L1 cache.

MULTIPROCESSOR AND MULTICORE ORGANIZATION

Multiprocessor Operating System

Multiprocessor Operating System refers to the use of two or more central processing units (CPU) within a single computer system. These multiple CPUs are in a close communication sharing the computer bus, memory and other peripheral devices. These systems are referred as tightly coupled systems.



Multiprocessing Architecture

Multiprocessing system is based on the symmetric multiprocessing model, in which each processor runs an identical copy of operating system and these copies communicate with each other. In this system processor is assigned a specific task. A master processor controls the system. This scheme defines a master-slave relationship.

These systems can save money in compare to single processor systems because the processors can share peripherals, power supplies and other devices. The main advantage of multiprocessor system is to get more work done in a shorter period of time. Moreover, multiprocessor systems prove more reliable in the situations of failure of one processor. In this situation, the system with multiprocessor will not halt the system; it will only slow it down.

In order to employ multiprocessing operating system effectively, the computer system must have the followings:

Motherboard Support:

A motherboard capable of handling multiple processors. This means additional sockets or slots for the extra chips and a chipset capable of handling the multiprocessing arrangement.

Processor Support:

Multiprocessor system supports the processes to run in parallel. Parallel processing is the ability of the CPU to simultaneously process incoming jobs. This becomes most important in computer system, as the CPU divides and conquers the jobs. Generally the

Locking system:

In order to provide safe access to the resources shared among multiple processors, they need to be protected by locking scheme. The purpose of a locking is to serialize accesses to the protected resource by multiple processors.

Shared data:

The continuous accesses to the shared data items by multiple processors (with one or more of them with data write) are serialized by the cache coherence protocol.

False sharing:

This form of contention arises when unrelated data items used by different processors are located next to each other in the memory and, therefore, share a single cache line: The effect of false sharing is the same as that of regular sharing bouncing of the cache line among several processors. Fortunately, once it is identified, false sharing can be easily eliminated by setting the memory layout of non-shared data.

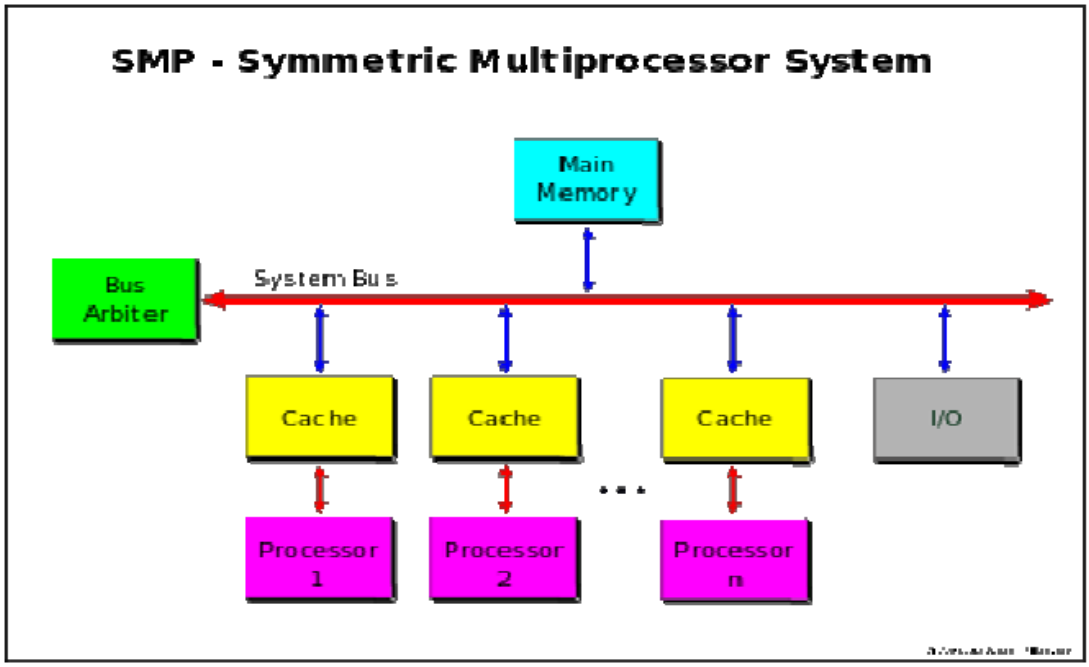
Types of Multiprocessors

There are mainly two types of multiprocessors i.e. symmetric and asymmetric multiprocessors. Details about them are as follows:

Symmetric Multiprocessors

In these types of systems, each processor contains a similar copy of the operating system and they all communicate with each other. All the processors are in a peer to peer relationship i.e. no master - slave relationship exists between them.

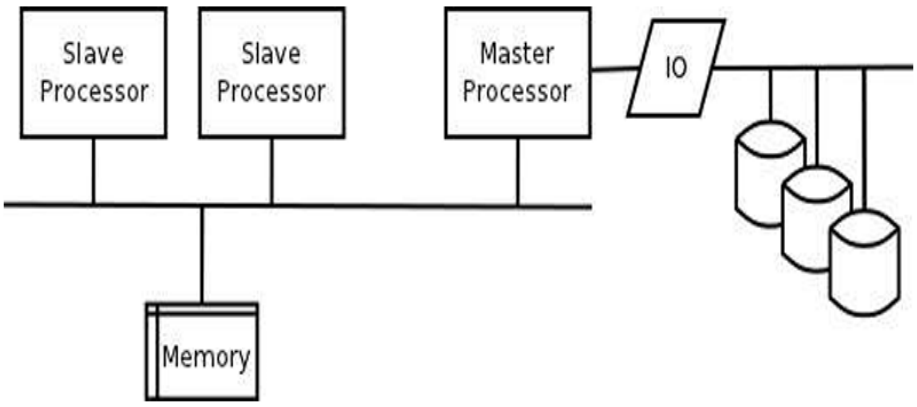
An example of the symmetric multiprocessing system is the Encore version of Unix for the Multimax Computer.



Asymmetric Multiprocessors

In asymmetric systems, each processor is given a predefined task. There is a master processor that gives instruction to all the other processors. Asymmetric multiprocessor system contains a master slave relationship.

Asymmetric multiprocessor was the only type of multiprocessor available before symmetric multiprocessors were created. Now also, this is the cheaper option.



Multicore Organization

A multicore processor is a single computing component comprised of two or more CPUs that read and execute the actual program instructions. The individual cores can execute multiple instructions in parallel, increasing the performance of software which has been written to take advantage of the unique architecture.

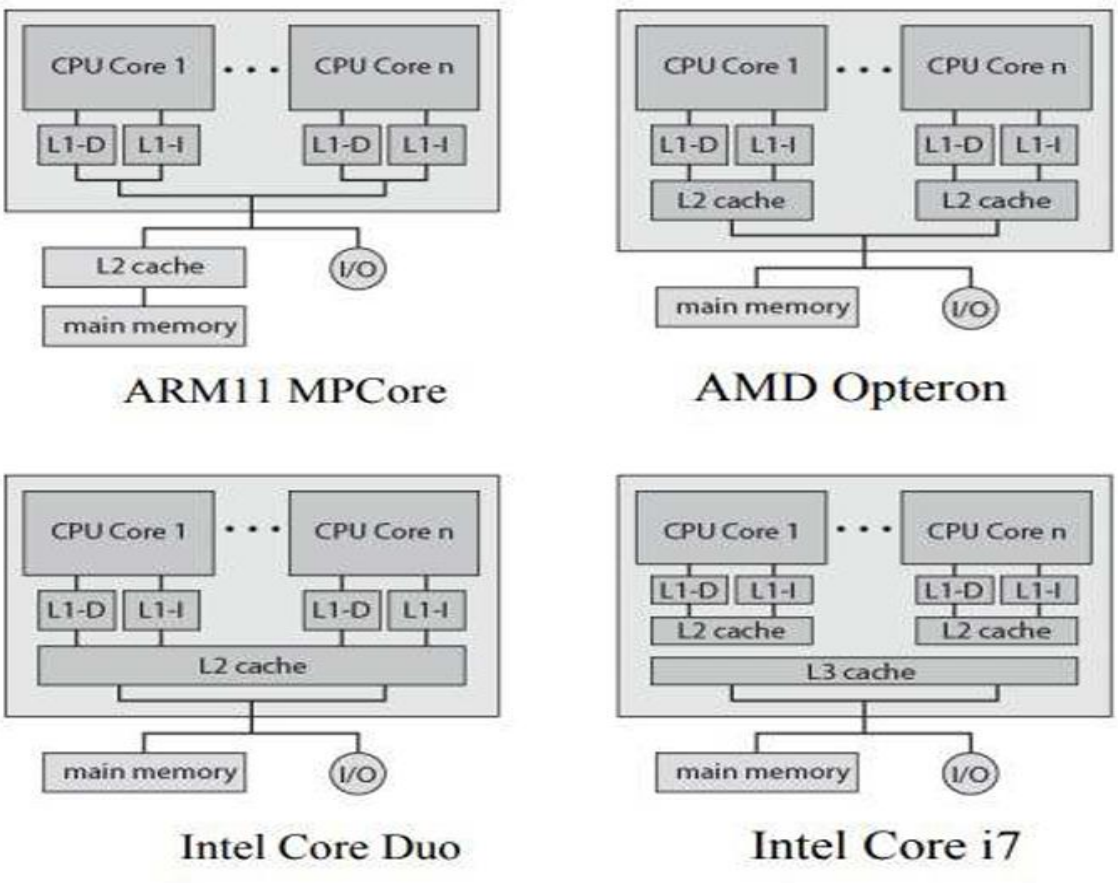
A multicore computer, also known as a chip multiprocessor, combines two or more processors (called cores) on a single piece of silicon (called a die). Typically, each core consists of all of the components of an independent processor, such as registers, ALU, pipeline hardware, and control unit, plus L1 instruction and data caches. In addition to the multiple cores, contemporary multicore chips also include L2 cache and, in some cases, L3 cache.

Main variable in a multicore organization:

Number of core processors on chip

Number of levels of cache on chip

Amount of shared cache



Multicore Organisation Alternatives

Private × shared L2 Cache

Advantages of shared L2 Cache

Constructive interference reduces overall miss rate

Data shared by multiple cores not replicated at cache level

With proper frame replacement algorithms mean amount of shared cache dedicated to each core is dynamic. Threads with less locality can have more cache

Easy inter-process communication through shared memory

Cache coherency confined to L1

Advantages of private L2 Cache

Dedicated L2 cache gives each core more rapid access

Advantages of shared L3 Cache

Shared L3 cache may also improve performance